



MYTHIC

BEVFormer Retraining Guide

May 2026

Document Version 1.0
SDK Version v26.05.0

1	BEVFormer.....	3
1.1	Variants.....	3
1.2	Folder Structure.....	3
1.3	Main Deltas from Original Implementation	3
1.4	Conversion Overview	4
1.5	Setup	4
1.5.1	Set up CUDA Dependencies	4
1.5.2	Dependencies Setup	4
1.5.3	Model Selection.....	4
1.5.4	Dataset Setup	5
1.6	Training FP32	5
1.7	Export FP32 PyTorch Model to ONNX	6
1.8	Structural Modifications	6
1.9	Conversion To Mythic	6
1.10	Mythic Retraining	7
1.11	Bake Snapshot Weights to ONNX Model (optional)	7
1.12	Post Processing	7
1.13	Eval Analog-Aware ONNX Model (Accuracy Simulation)	7
1.14	Compiler Artifacts	8
1.15	Generating Inference Videos (fp32, onnx, analog-aware)	8
1.16	ONNX Model Inputs and Outputs.....	9

MYTHIC

1 BEVFormer

- BEVFormer paper: [BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers](#)
- Original BEVFormer repo: <https://github.com/fundamentalvision/BEVFormer/>
- Onnx Export support code: <https://github.com/AXERA-TECH/bevformer.axera>

1.1 Variants

Integrated

- BEVFormer Tiny @ 1600x900 resolution

Not Yet Integrated

- BEVFormer Base @ 1600x900 resolution

1.2 Folder Structure

Primary directories/files

```
├ conversion_steps.py # MUNC-based conversion steps
├ bevformer_lib/ # primarily third-party code to implement/define BEVFormer
│   ├── projects/
│   │   ├── configs/ # MMCV-based BEVFormer configs (defines network structure)
│   │   └ mmdet3d_plugin/
│   │       └ bevformer/ # primary BEVFormer module implementations
│   └ custom_utils/ # support code added for inference and visualization
└ bevformer_inference.py # generalized BEVFormer inference script (works
  with .pth, .onnx, and TorchNet)
```

1.3 Main Deltas from Original Implementation

- Dependency modernization
 - Python: 3.8 -> >=3.10
 - mmcv-full: 1.4.0 -> modified 1.7.2
 - mmdet3d: 0.17.1 -> modified 1.0.0rc6
 - torch: 1.9.1 -> >=2
 - numpy: 1.19.5 -> >=2
 - CUDA: 11 -> 12
 - other minor dependencies
- ONNX Export support
 - Most of the logic comes from <https://github.com/AXERA-TECH/bevformer.axera> with some modifications (some of which are informed by https://github.com/DerryHub/BEVFormer_tensorrt)
- BEVFormer Tiny support of 1600x900 resolution

MYTHIC

1.4 Conversion Overview

Steps

- Download and set up a dataset. For example, nuScenes. Note this is not managed via model zoo.
- `generate_nuscenes_annotations` - This needs to be done once.
- `train_torch_fp` - Perform regular fp32-based BEVFormer training.
- `to_onnx` - Export BEVFormer PyTorch model to ONNX.
- `to_structural` - Convert fp32 ONNX model to intermediate "structural" graph.
- `to_training` - Convert structural graph to quantized analog-aware graph suitable for retraining.
- `train` - Perform analog-aware retraining.
- `to_acm` - Convert the retrained graph to an evaluation "acm" ONNX graph.
- `create_artifact` - Create an artifact for Mythic's compiler

Note: each step takes the subsequent step's output as its input

1.5 Setup

1.5.1 Set up CUDA Dependencies

Note: some of BEVFormer's dependencies require building from source, hence the extra CUDA libraries

```
apt-get install -y cuda-nvcc-12-9 cuda-libraries-dev-12-9
export CUDA_HOME=/usr/local/cuda-12.9 # or other install location as applicable
```

1.5.2 Dependencies Setup

```
uv venv --python=3.12 venv
export UV_PROJECT_ENVIRONMENT=venv
uv sync --extra 'bevformer,acm-m2000,infer,wandb'
# 'wandb' extra can be omitted if not using Weights and Biases
```

1.5.3 Model Selection

Activate the virtual environment and configure environment variables

```
source ./scripts/bevformer/bevformer-tiny-1600x900.env
```

Configuration

See the following for the original MMCV-based configs for the model

MYTHIC

- `mythic/model_zoo/bevformer/bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py`

See files in `configs/bevformer` for Mythic-specific configuration of BEVFormer conversion and retraining

1.5.4 Dataset Setup

Follow the setup instructions from <https://www.nuscenes.org/nuscenes> / <https://github.com/nutonomy/nuscenes-devkit> to download and set up the dataset folder structures, including the can bus extension.

Generate Dataset Annotation Files

Configure the dataset and annotations paths in the BEVFormer config

```
# configs/bevformer/bevformer_tiny.yaml
nuscenes_root: /path/to/downloaded/nuscenes
nuscenes_data_annotation_root: /path/to/generate/nuscenes/annotations/to
pytorch_checkpoint_root: /path/to/pytorch/fp32/pth/files
```

Where:

- `nuscenes_root` : The path to the dataset downloaded above
 - Overrides the `data_root` path in `mythic/model_zoo/bevformer/bevformer_lib/projects/configs/bevformer` configs
- `nuscenes_data_annotation_root` : Location to generate .pkl files that serve as preprocessed nuScenes indexes/caches for BEVFormer. They flatten the nuScenes JSON tables plus CAN bus extension into the exact structure BEVFormer expects
 - Overrides the `data_annotation_root` path in `mythic/model_zoo/bevformer/bevformer_lib/projects/configs/bevformer` configs
 - *Note: if you have other nuScenes-based projects, the annotations generated are not necessarily transferable between projects. As such, you'll likely want to set the `nuscenes_data_annotation_root` to a BEVFormer-specific directory*
- `pytorch_checkpoint_root` : Any downloaded PyTorch fp32 checkpoints (which can be used at the starting weights when exporting to ONNX)

Run generation script:

```
python3 scripts/common/convert_model.py steps=generate_nuscenes_annotations
```

Note: This can take a while (1 hr+)

1.6 Training FP32

If you aren't starting conversion from a pretrained `.pth` checkpoint, you'll likely want to train in fp32 first.

MYTHIC

Note: you can skip the initial fp32 training if you want to use initial weights (for BEVFormer Tiny, the ResNet backbone is initialized from the pretrained `torchvision://resnet50`, and the transformer head is untrained).

```
# 1 GPU
python3 scripts/common/convert_model.py steps=train_torch_fp
# - or -
# Multiple GPUs (change the --nproc-per-node value accordingly)
torchrun --standalone --nproc-per-node=4 scripts/common/convert_model.py
steps=train_torch_fp
```

1.7 Export FP32 PyTorch Model to ONNX

Exports the PyTorch model to an ONNX model.

```
python3 scripts/common/convert_model.py steps=to_onnx
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::to_onnx`

Output: `data/fp32-1600x900.onnx`

1.8 Structural Modifications

- On chip vs off chip
 - The ResNet image backbone is placed on chip
 - The transformer head is kept digital
- Light constant folding is performed to simplify the model graph without folding subgraphs that should remain present and trainable

```
python3 scripts/common/convert_model.py steps=to_structural
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::to_structural`

Output: `data/structural-1600x900.onnx`

1.9 Conversion To Mythic

Converts the structural graph to a quantized analog-aware graph suitable for retraining.

```
python3 scripts/common/convert_model.py steps=to_training
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::to_training`

Output: `data/mythic-1600x900.onnx`

MYTHIC

1.10 Mythic Retraining

Analog-aware retraining.

```
python3 scripts/common/convert_model.py steps=train
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::train`

Training Configuration: `configs/bevformer/training_config/tiny_1600x900.yaml`

Outputs:

- `data/trained-1600x900.onnx` (trained onnx model at the end of training)
- `data/bevformer_training/1600x900/<training-run-name>/epoch_*.pth` (per-epoch snapshots)

1.11 Bake Snapshot Weights to ONNX Model (optional)

While the above Mythic retraining will save the trained weights back to `data/trained-1600x900.onnx` at the end of training, you can use this step to save mid-training checkpoints back to an onnx model.

```
python3 scripts/common/convert_model.py steps=bake_snapshot_weights_to_onnx
```

Step Configuration: `configs/bevformer/
bevformer_tiny.yaml::bake_snapshot_weights_to_onnx`

Output: `data/trained-1600x900-<custom-label>.onnx`

1.12 Post Processing

Removes training-oriented graph nodes/structures (e.g. dropout). Optionally performs constant folding to further simplify the model and optimize for TorchNet inference/eval speed (not enabled by default).

```
python3 scripts/common/convert_model.py steps=post_retraining_simplification
```

Step Configuration: `configs/bevformer/
bevformer_tiny.yaml::post_retraining_simplification`

Output: `data/post-training-processed-1600x900.onnx`

1.13 Eval Analog-Aware ONNX Model (Accuracy Simulation)

This runs the eval on an analog-aware Mythic model (by default the `trained-1600x900.onnx` model, but will also work with other BEVFormer ONNX models). Use this to collect mAP, NDS, mATE, mASE, mAOE, mAVE, mAAE (as well as the per-class level metrics).

MYTHIC

```
python3 scripts/common/convert_model.py steps=eval_mythic_model
```

```
python3 scripts/common/convert_model.py steps=eval_mythic_model trained_model=override/  
path/to/model.onnx
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::eval_mythic_model`

Output: Eval metrics

1.14 Compiler Artifacts

Creates an artifact for Mythic's compiler, going through an ACM intermediate step

```
python3 scripts/common/convert_model.py steps=to_acm
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::to_acm`

Output: `data/acm-1600x900.onnx`

```
python3 scripts/common/convert_model.py steps=create_artifact
```

Step Configuration: `configs/bevformer/bevformer_tiny.yaml::create_artifact`

Output: `data/compiler_ready_artifact-1600x900.tar.gz`

1.15 Generating Inference Videos (fp32, onnx, analog-aware)

For full options see `mythic/model_zoo/bevformer/bevformer_inference.py --help`

Examples below:

FP32

```
./bevformer_inference.py pytorch \  
  '/path/to/checkpoint/epoch_<#>.pth' \  
  bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py \  
  --end-scene=5 --map-bev --data-type=sweeps
```

Supports:

- `.pth` checkpoints

ONNX

MYTHIC

```
./bevformer_inference.py onnx \  
  '/path/to/mythic-1600x900.onnx' \  
  bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py \  
  --end-scene=5 --map-bev --data-type=sweeps
```

Supports:

- fp32-1600x900.onnx
- structural-1600x900.onnx

Analog Aware

```
# From checkpoint  
./bevformer_inference.py torchnet \  
  '/path/to/mythic-1600x900.onnx' \  
  bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py \  
  --checkpoint '/path/to/checkpoint/epoch_<number>.pth' \  
  --end-scene=5 --map-bev --data-type=sweeps  
  
# Trained  
./bevformer_inference.py torchnet \  
  '/path/to/trained-1600x900.onnx' \  
  bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py \  
  --end-scene=5 --map-bev --data-type=sweeps
```

Supports:

- mythic-1600x900.onnx
- trained-1600x900.onnx
- post-training-processed-1600x900.onnx

Ground Truth Reference

```
./bevformer_inference.py ground-truth \  
  bevformer_lib/projects/configs/bevformer/bevformer_tiny-1600x900.py \  
  --end-scene=5
```

1.16 ONNX Model Inputs and Outputs

Where:

- B : batch size
- W : image input width in pixels
 - 1600

MYTHIC

- `H` : image input height in pixels
 - 900
- `C` : image channels
 - 3
- `N` : number of cameras
 - 6
- `bev_w` : BEV space columns
 - Tiny: 50
- `bev_h` : BEV space rows
 - Tiny: 50
- `embed_dims` : embedding dimension
 - 256
- `D` : number of decoder layers
 - 6
- `Q` : number of object queries
 - e.g. 900
- `num_classes` : number of object classes
 - 10
- `code_size` : number of fields used to define 3D bounding boxes
 - 10 (i.e. `cx,cy,w,l,cz,h,sin,cos,vx,vy`)

Inputs

Input	Shape	Description
<code>img</code>	<code>[B , N , C , H , W]</code>	The 6 camera inputs
<code>can_bus</code>	<code>[B , 18]</code>	18 data fields from the can bus (e.g. position, orientation, velocity, acceleration)
<code>lidar2img</code>	<code>[B , N , 4 , 4]</code>	Despite the "lidar" this is a per-camera projection matrix to convert from the common 3D coordinate space (canonically lidar-centered on nuScenes) to each camera's image (pixel) coordinates

MYTHIC

Input	Shape	Description
<code>prev_be</code>	<code>[B , bev_h * bev_w , embed_dims]</code>	BEV feature state carried forward between frames
<code>use_prevbev</code>	<code>[1]</code>	Used as a boolean flag of whether or not to use the <code>prev_be</code> (namely at the start of a new scene and in retraining at the start of constructing historical BEV from a sequence of samples)

Outputs

Output	Shape	Description
<code>outputs_coords</code>	<code>[B , D , Q , code_size]</code>	Per decoder layer regression for each object query as 3D bounding boxes (see <code>projects/mmdet3d_plugin/core/bbox/util.py::denormalize_bbox</code> , channels are <code>(cx, cy, log w, log l, cz, log h, sin θ, cos θ, vx, vy)</code>). Only the last decoder layer is used for inference.
<code>bev_embed</code>	<code>[B , bev_h * bev_w , embed_dims]</code>	Encoder output: the BEV feature grid, used both as the decoder's K/V and as the <code>prev_be</code> pass to the next frame.
<code>outputs_classes</code>	<code>[B , D , Q , num_classes]</code>	Per decoder layer class logits per query. Only the last decoder layer is used at inference.